

INDIAN PREMIER LEAGUE (IPL) — DATA ANALYSIS

Exploratory Data Analysis & Visualization Using Python

Project By: Harsh Pandey **Project Guide:** Mohit Sir | CETPA Infotech

Tools & Technologies:  Python •  NumPy •  Pandas •  Matplotlib •  Seaborn •  Jupyter Notebook •  VS Code

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
In [2]: matches = pd.read_csv("../data/matches.csv")
deliveries = pd.read_csv("../data/deliveries.csv")

print("Matches:", matches.shape)
print("Deliveries:", deliveries.shape)
```

Matches: (1095, 20)

Deliveries: (260920, 17)

```
In [3]: matches.head()
```

```
Out[3]:
```

	id	season	city	date	match_type	player_of_match	venue	team1	team2
0	335982	2007/08	Bangalore	2008-04-18	League	BB McCullum	M Chinnaswamy Stadium	Royal Challengers Bangalore	Kolkata Knight Riders
1	335983	2007/08	Chandigarh	2008-04-19	League	MEK Hussey	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab	Chennai Super Kings
2	335984	2007/08	Delhi	2008-04-19	League	MF Maharoo	Feroz Shah Kotla	Delhi Daredevils	Rajasthan Royals
3	335985	2007/08	Mumbai	2008-04-20	League	MV Boucher	Wankhede Stadium	Mumbai Indians	Royal Challengers Bangalore
4	335986	2007/08	Kolkata	2008-04-20	League	DJ Hussey	Eden Gardens	Kolkata Knight Riders	Deccan Chargers

```
In [4]: deliveries.head()
```

Out[4]:	match_id	inning	batting_team	bowling_team	over	ball	batter	bowler	non_striker	batsman_runs
0	335982	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	1	SC Ganguly	P Kumar	BB McCullum	0
1	335982	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	2	BB McCullum	P Kumar	SC Ganguly	0
2	335982	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	3	BB McCullum	P Kumar	SC Ganguly	0
3	335982	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	4	BB McCullum	P Kumar	SC Ganguly	0
4	335982	1	Kolkata Knight Riders	Royal Challengers Bangalore	0	5	BB McCullum	P Kumar	SC Ganguly	0

In [5]: `matches.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 1095 entries, 0 to 1094
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    1095 non-null  int64
1   season               1095 non-null  str
2   city                 1044 non-null  str
3   date                 1095 non-null  str
4   match_type           1095 non-null  str
5   player_of_match      1090 non-null  str
6   venue                1095 non-null  str
7   team1                1095 non-null  str
8   team2                1095 non-null  str
9   toss_winner          1095 non-null  str
10  toss_decision        1095 non-null  str
11  winner               1090 non-null  str
12  result               1095 non-null  str
13  result_margin        1076 non-null  float64
14  target_runs          1092 non-null  float64
15  target_overs         1092 non-null  float64
16  super_over           1095 non-null  str
17  method               21 non-null    str
18  umpire1              1095 non-null  str
19  umpire2              1095 non-null  str
dtypes: float64(3), int64(1), str(16)
memory usage: 171.2 KB
```

In [6]: `matches.shape`
`deliveries.shape`

Out[6]: (260920, 17)

```
In [7]: print("MATCHES COLUMNS:")
        print(matches.columns.tolist())

        print("\nDELIVERIES COLUMNS:")
        print(deliveries.columns.tolist())
```

MATCHES COLUMNS:

```
['id', 'season', 'city', 'date', 'match_type', 'player_of_match', 'venue', 'team1', 'team2', 'toss_winner', 'toss_decision', 'winner', 'result', 'result_margin', 'target_runs', 'target_overs', 'super_over', 'method', 'umpire1', 'umpire2']
```

DELIVERIES COLUMNS:

```
['match_id', 'inning', 'batting_team', 'bowling_team', 'over', 'ball', 'batter', 'bowler', 'non_striker', 'batsman_runs', 'extra_runs', 'total_runs', 'extras_type', 'is_wicket', 'player_dismissed', 'dismissal_kind', 'fielder']
```

```
In [8]: matches.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1095 entries, 0 to 1094
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    1095 non-null   int64
1   season                1095 non-null   str
2   city                  1044 non-null   str
3   date                  1095 non-null   str
4   match_type            1095 non-null   str
5   player_of_match       1090 non-null   str
6   venue                 1095 non-null   str
7   team1                 1095 non-null   str
8   team2                 1095 non-null   str
9   toss_winner           1095 non-null   str
10  toss_decision         1095 non-null   str
11  winner                 1090 non-null   str
12  result                 1095 non-null   str
13  result_margin         1076 non-null   float64
14  target_runs           1092 non-null   float64
15  target_overs          1092 non-null   float64
16  super_over            1095 non-null   str
17  method                 21 non-null     str
18  umpire1               1095 non-null   str
19  umpire2               1095 non-null   str
dtypes: float64(3), int64(1), str(16)
memory usage: 171.2 KB
```

```
In [9]: print("Missing Values in Matches:")
        print(matches.isnull().sum())
```

Missing Values in Matches:

id	0
season	0
city	51
date	0
match_type	0
player_of_match	5
venue	0
team1	0
team2	0
toss_winner	0
toss_decision	0
winner	5
result	0
result_margin	19
target_runs	3
target_overs	3
super_over	0
method	1074
umpire1	0
umpire2	0

dtype: int64

```
In [10]: print("Missing Values in Deliveries:")  
print(deliveries.isnull().sum())
```

Missing Values in Deliveries:

match_id	0
inning	0
batting_team	0
bowling_team	0
over	0
ball	0
batter	0
bowler	0
non_striker	0
batsman_runs	0
extra_runs	0
total_runs	0
extras_type	246795
is_wicket	0
player_dismissed	247970
dismissal_kind	247970
fielder	251566

dtype: int64

```
In [11]: print("Duplicate rows in Matches:", matches.duplicated().sum())  
  
print("Duplicate rows in Deliveries:", deliveries.duplicated().sum())
```

Duplicate rows in Matches: 0

Duplicate rows in Deliveries: 0

```
In [12]: matches.isnull().sum()
```

```
Out[12]: id          0
         season      0
         city        51
         date         0
         match_type   0
         player_of_match  5
         venue        0
         team1        0
         team2        0
         toss_winner   0
         toss_decision  0
         winner        5
         result        0
         result_margin 19
         target_runs    3
         target_overs   3
         super_over     0
         method       1074
         umpire1        0
         umpire2        0
         dtype: int64
```

```
In [13]: matches_clean = matches.copy()
         deliveries_clean = deliveries.copy()

         print("Copies created successfully!")
```

Copies created successfully!

```
In [14]: df = matches_clean.copy()
         print('df initialized from matches_clean')
```

df initialized from matches_clean

```
In [15]: matches_clean.drop_duplicates(inplace=True)
         deliveries_clean.drop_duplicates(inplace=True)

         print("Duplicate rows removed!")
```

Duplicate rows removed!

```
In [16]: matches_clean.drop_duplicates(inplace=True)
         deliveries_clean.drop_duplicates(inplace=True)

         print("Duplicate rows removed!")
```

Duplicate rows removed!

```
In [17]: matches_clean.isnull().sum()
```

```
Out[17]: id          0
         season      0
         city        51
         date         0
         match_type   0
         player_of_match 5
         venue        0
         team1        0
         team2        0
         toss_winner   0
         toss_decision 0
         winner        5
         result        0
         result_margin 19
         target_runs    3
         target_overs  3
         super_over     0
         method       1074
         umpire1       0
         umpire2       0
         dtype: int64
```

```
In [18]: print(matches_clean.columns.tolist())
```

```
['id', 'season', 'city', 'date', 'match_type', 'player_of_match', 'venue', 'team1', 'team2', 'toss_winner', 'toss_decision', 'winner', 'result', 'result_margin', 'target_runs', 'target_overs', 'super_over', 'method', 'umpire1', 'umpire2']
```

```
In [19]: matches_clean["date"] = pd.to_datetime(
         matches_clean["date"],
         errors="coerce"
         )
```

```
In [20]: matches_clean["date"].head()
```

```
Out[20]: 0    2008-04-18
         1    2008-04-19
         2    2008-04-19
         3    2008-04-20
         4    2008-04-20
         Name: date, dtype: datetime64[us]
```

```
In [21]: matches_clean.dtypes
```

```
Out[21]: id int64
season str
city str
date datetime64[us]
match_type str
player_of_match str
venue str
team1 str
team2 str
toss_winner str
toss_decision str
winner str
result str
result_margin float64
target_runs float64
target_overs float64
super_over str
method str
umpire1 str
umpire2 str
dtype: object
```

```
In [22]: deliveries_clean.dtypes
```

```
Out[22]: match_id int64
inning int64
batting_team str
bowling_team str
over int64
ball int64
batter str
bowler str
non_striker str
batsman_runs int64
extra_runs int64
total_runs int64
extras_type str
is_wicket int64
player_dismissed str
dismissal_kind str
fielder str
dtype: object
```

```
In [23]: print(matches_clean["team1"].unique())
```

```
<StringArray>
['Royal Challengers Bangalore', 'Kings XI Punjab',
 'Delhi Daredevils', 'Mumbai Indians',
 'Kolkata Knight Riders', 'Rajasthan Royals',
 'Deccan Chargers', 'Chennai Super Kings',
 'Kochi Tuskers Kerala', 'Pune Warriors',
 'Sunrisers Hyderabad', 'Gujarat Lions',
 'Rising Pune Supergiants', 'Rising Pune Supergiant',
 'Delhi Capitals', 'Punjab Kings',
 'Lucknow Super Giants', 'Gujarat Titans',
 'Royal Challengers Bengaluru']
Length: 19, dtype: str
```

```
In [24]: print(matches_clean["team1"].unique())
```

```
<StringArray>
['Royal Challengers Bangalore', 'Kings XI Punjab',
 'Delhi Daredevils', 'Mumbai Indians',
 'Kolkata Knight Riders', 'Rajasthan Royals',
 'Deccan Chargers', 'Chennai Super Kings',
 'Kochi Tuskers Kerala', 'Pune Warriors',
 'Sunrisers Hyderabad', 'Gujarat Lions',
 'Rising Pune Supergiants', 'Rising Pune Supergiant',
 'Delhi Capitals', 'Punjab Kings',
 'Lucknow Super Giants', 'Gujarat Titans',
 'Royal Challengers Bengaluru']
Length: 19, dtype: str
```

```
In [25]: total_matches = matches_clean.shape[0]

print("Total IPL Matches:", total_matches)
```

Total IPL Matches: 1095

```
In [26]: print(matches_clean["season"].unique())

<StringArray>
['2007/08', '2009', '2009/10', '2011', '2012', '2013', '2014',
 '2015', '2016', '2017', '2018', '2019', '2020/21', '2021',
 '2022', '2023', '2024']
Length: 17, dtype: str
```

```
In [27]: team_wins = matches_clean["winner"].value_counts()

print(team_wins)
```

```
winner
Mumbai Indians          144
Chennai Super Kings     138
Kolkata Knight Riders   131
Royal Challengers Bangalore 116
Rajasthan Royals        112
Kings XI Punjab         88
Sunrisers Hyderabad     88
Delhi Daredevils        67
Delhi Capitals           48
Deccan Chargers         29
Gujarat Titans          28
Punjab Kings            24
Lucknow Super Giants    24
Gujarat Lions           13
Pune Warriors           12
Rising Pune Supergiant  10
Royal Challengers Bengaluru 7
Kochi Tuskers Kerala    6
Rising Pune Supergiants 5
Name: count, dtype: int64
```

```
In [28]: plt.figure(figsize=(12, 6))

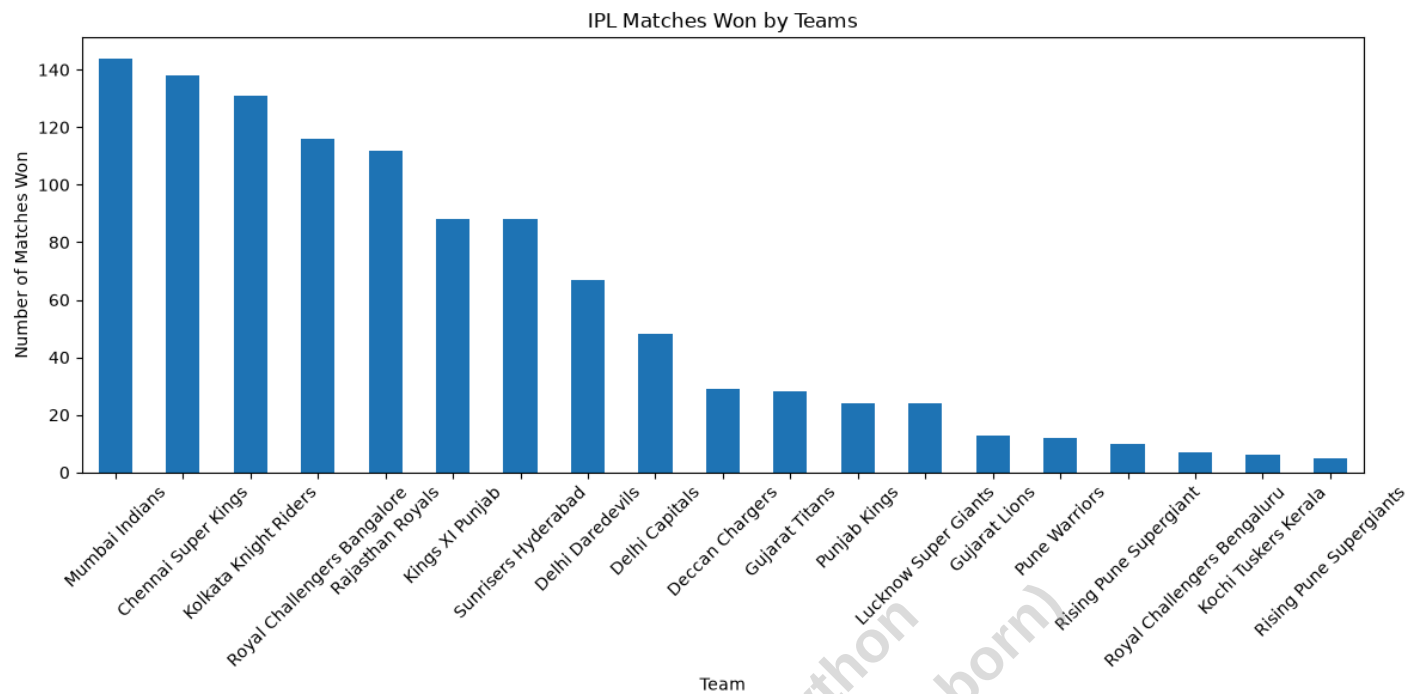
team_wins.plot(kind="bar")

plt.title("IPL Matches Won by Teams")
plt.xlabel("Team")
plt.ylabel("Number of Matches Won")

plt.xticks(rotation=45)

plt.tight_layout()
```

```
plt.show()
```



```
In [29]: print(deliveries_clean.columns.tolist())
```

```
['match_id', 'inning', 'batting_team', 'bowling_team', 'over', 'ball', 'batter', 'bowler', 'non_striker', 'batsman_runs', 'extra_runs', 'total_runs', 'extras_type', 'is_wicket', 'player_dismissed', 'dismissal_kind', 'fielder']
```

```
In [30]: batsman_column = "batter"
```

```
In [31]: batsman_column = "batsman"
```

```
In [32]: top_batsmen = (
    deliveries_clean.groupby(batsman_column)[batsman_runs]
    .sum()
    .sort_values(ascending=False)
    .head(10)
)

print(top_batsmen)
```

```
batter
V Kohli      8014
S Dhawan     6769
RG Sharma    6630
DA Warner    6567
SK Raina     5536
MS Dhoni     5243
AB de Villiers 5181
CH Gayle     4997
RV Uthappa   4954
KD Karthik   4843
Name: batsman_runs, dtype: int64
```

```
In [33]: plt.figure(figsize=(12, 6))

top_batsmen.plot(kind="bar")

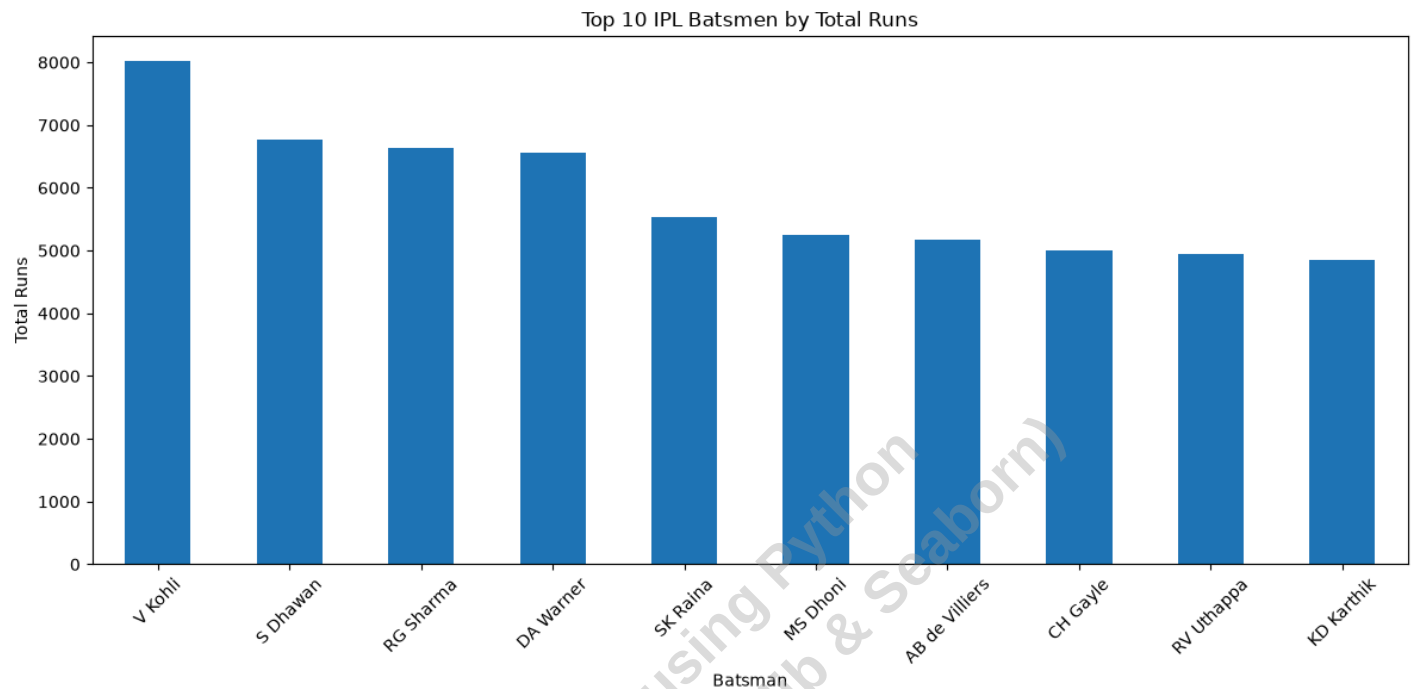
plt.title("Top 10 IPL Batsmen by Total Runs")
plt.xlabel("Batsman")
```

```
plt.ylabel("Total Runs")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```



```
In [34]: print(deliveries_clean.columns.tolist())

['match_id', 'inning', 'batting_team', 'bowling_team', 'over', 'ball', 'batter', 'bowler', 'non_striker', 'batsman_runs', 'extra_runs', 'total_runs', 'extras_type', 'is_wicket', 'player_dismissed', 'dismissal_kind', 'fielder']
```

```
In [35]: deliveries_clean["dismissal_kind"].value_counts()
```

```
Out[35]: dismissal_kind
caught          8063
bowled          2212
run out         1114
lbw              800
caught and bowled 367
stumped         358
retired hurt     15
hit wicket       15
obstructing the field 3
retired out      3
Name: count, dtype: int64
```

```
In [36]: valid_wickets = deliveries_clean[
    ~deliveries_clean["dismissal_kind"].isin(
        ["run out", "retired hurt", "obstructing the field"]
    )
]
```

```
In [37]: top_bowlers = (
    valid_wickets.groupby("bowler")
    .size()
    .sort_values(ascending=False)
    .head(10)
)
```

```
print(top_bowlers)
```

```
bowler
R Ashwin      4661
SP Narine     4126
B Kumar       4046
PP Chawla     3886
RA Jadeja     3886
YS Chahal     3620
Harbhajan Singh 3485
A Mishra      3435
DJ Bravo      3272
UT Yadav      3171
dtype: int64
```

```
In [38]: plt.figure(figsize=(12, 6))

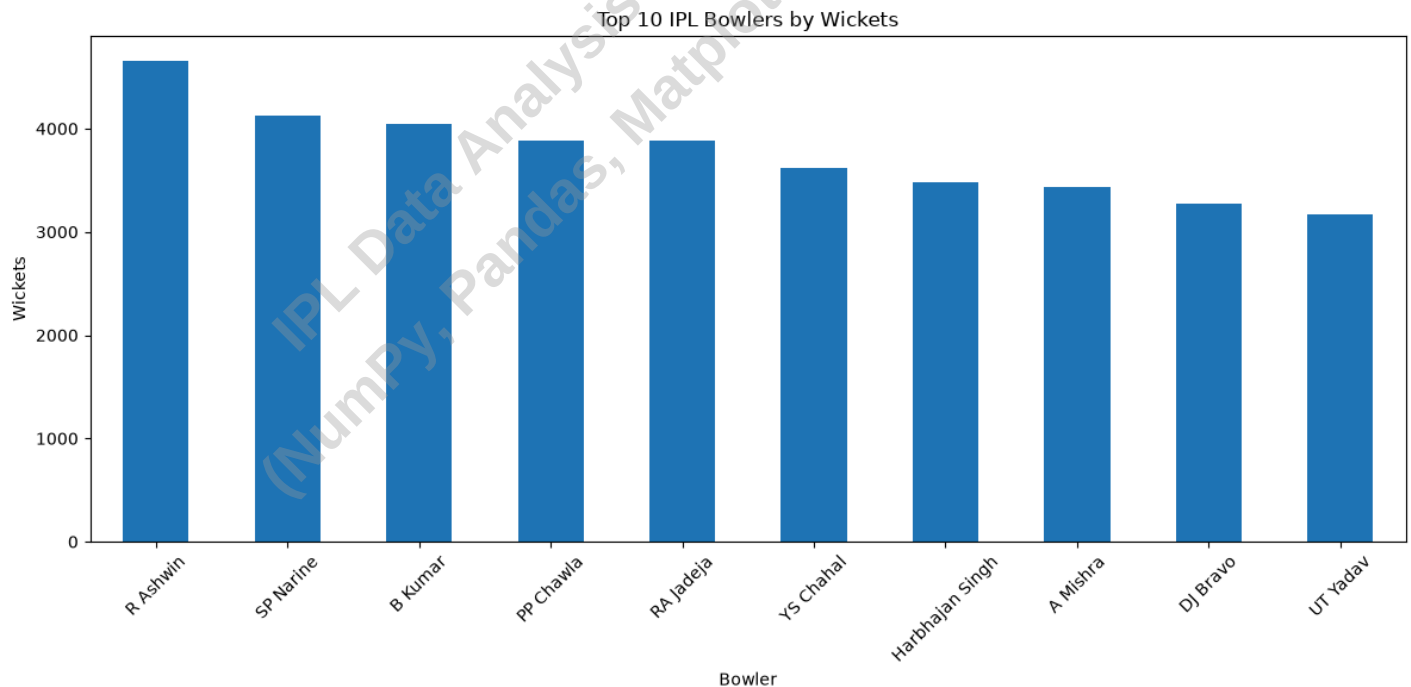
top_bowlers.plot(kind="bar")

plt.title("Top 10 IPL Bowlers by Wickets")
plt.xlabel("Bowler")
plt.ylabel("Wickets")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```



```
In [39]: # Step 28: Player of the Match Analysis
```

```
player_of_match_counts = (
    matches_clean["player_of_match"]
    .dropna()
    .value_counts()
    .head(10)
)

player_of_match_counts
```

```
Out[39]: player_of_match
AB de Villiers    25
CH Gayle          22
RG Sharma         19
DA Warner         18
V Kohli           18
MS Dhoni          17
SR Watson         16
YK Pathan         16
RA Jadeja         16
SP Narine         15
Name: count, dtype: int64
```

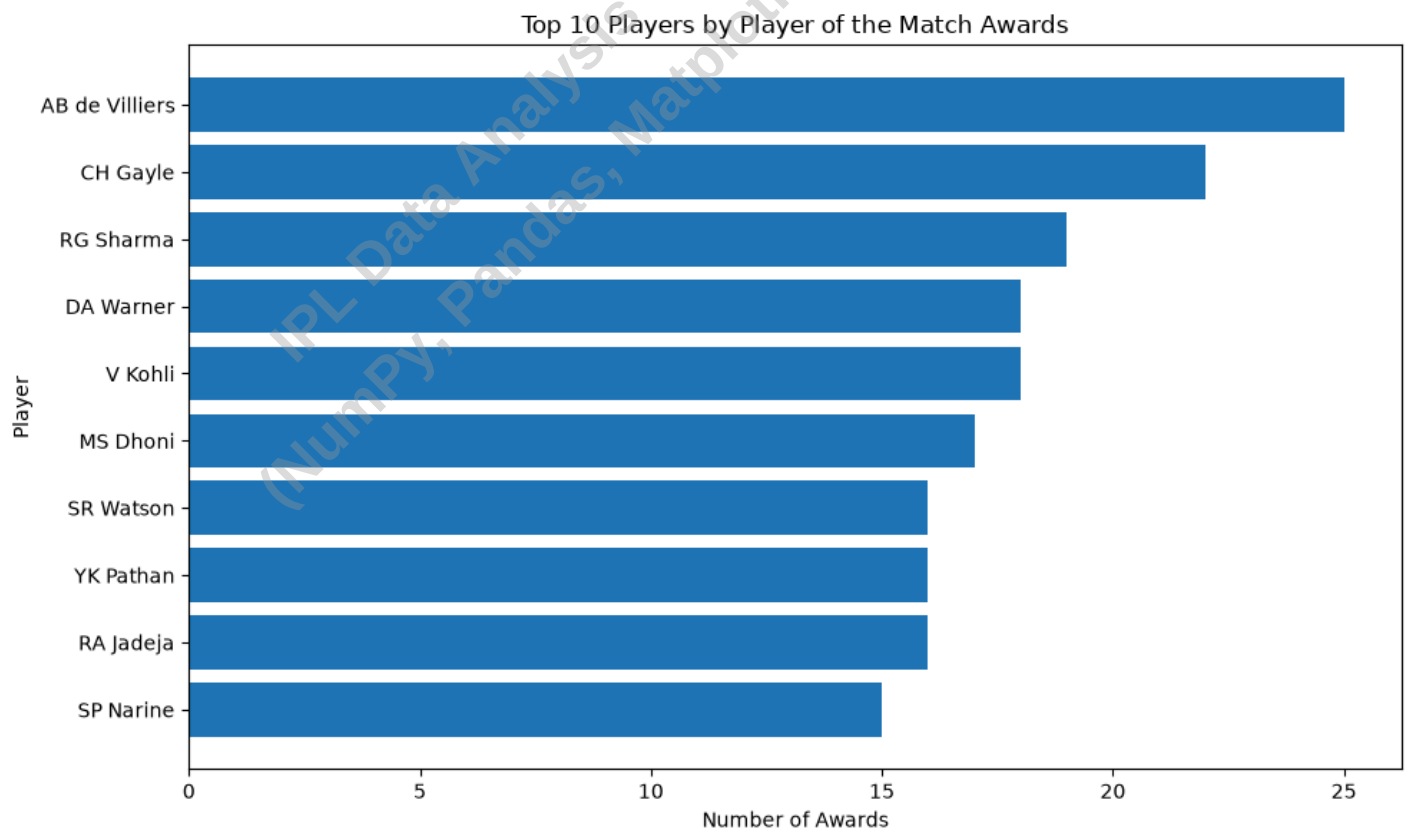
```
In [40]: import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 6))
```

```
plt.barh(
    player_of_match_counts.index[::-1],
    player_of_match_counts.values[::-1]
)
```

```
plt.title('Top 10 Players by Player of the Match Awards')
plt.xlabel('Number of Awards')
plt.ylabel('Player')
```

```
plt.tight_layout()
plt.show()
```



```
In [41]: pom_df = player_of_match_counts.reset_index()

pom_df.columns = ['Player', 'Player of the Match Awards']

pom_df
```

Out[41]:

	Player	Player of the Match Awards
0	AB de Villiers	25
1	CH Gayle	22
2	RG Sharma	19
3	DA Warner	18
4	V Kohli	18
5	MS Dhoni	17
6	SR Watson	16
7	YK Pathan	16
8	RA Jadeja	16
9	SP Narine	15

In [42]:

```
player_of_match_counts = (  
    df["player_of_match"]  
    .dropna()  
    .value_counts()  
)  
  
top_pom_player = player_of_match_counts.idxmax()  
top_pom_awards = player_of_match_counts.max()  
  
print(f"Player with the most Player of the Match awards: {top_pom_player}")  
print(f"Number of awards: {top_pom_awards}")  
  
top_pom_awards = player_of_match_counts.max()  
  
print(f"Player with the most Player of the Match awards: {top_pom_player}")  
  
print(f"Number of awards: {top_pom_awards}")
```

Player with the most Player of the Match awards: AB de Villiers
Number of awards: 25
Player with the most Player of the Match awards: AB de Villiers
Number of awards: 25

Insight

The Player of the Match analysis identifies the players who have produced the greatest individual match-winning impact across IPL seasons.

The player with the highest number of Player of the Match awards demonstrates a strong record of match-defining performances.

The analysis also highlights the consistency and impact of leading IPL players across different seasons.

In [43]:

```
import pandas as pd  
  
df = pd.read_csv("../data/matches.csv")
```

In [44]:

```
print(type(df))
```

<class 'pandas.DataFrame'>

In [45]: `# Step 29: Toss Impact Analysis`

```
df['toss_match_result'] = df.apply(  
    lambda row: 'Won Match' if row['toss_winner'] == row['winner']  
    else 'Lost Match',  
    axis=1  
)  
  
df['toss_match_result'].value_counts()
```

Out[45]:

toss_match_result	
Won Match	554
Lost Match	541

Name: count, dtype: int64

In [46]: `toss_impact = df['toss_match_result'].value_counts()`

```
toss_win_percentage = (  
    toss_impact['Won Match'] / toss_impact.sum()  
) * 100  
  
print(f"Toss winner also won the match: {toss_win_percentage:.2f}%")
```

Toss winner also won the match: 50.59%

In [47]: `toss_impact_df = toss_impact.reset_index()`

```
toss_impact_df.columns = ['Result', 'Matches']  
  
toss_impact_df
```

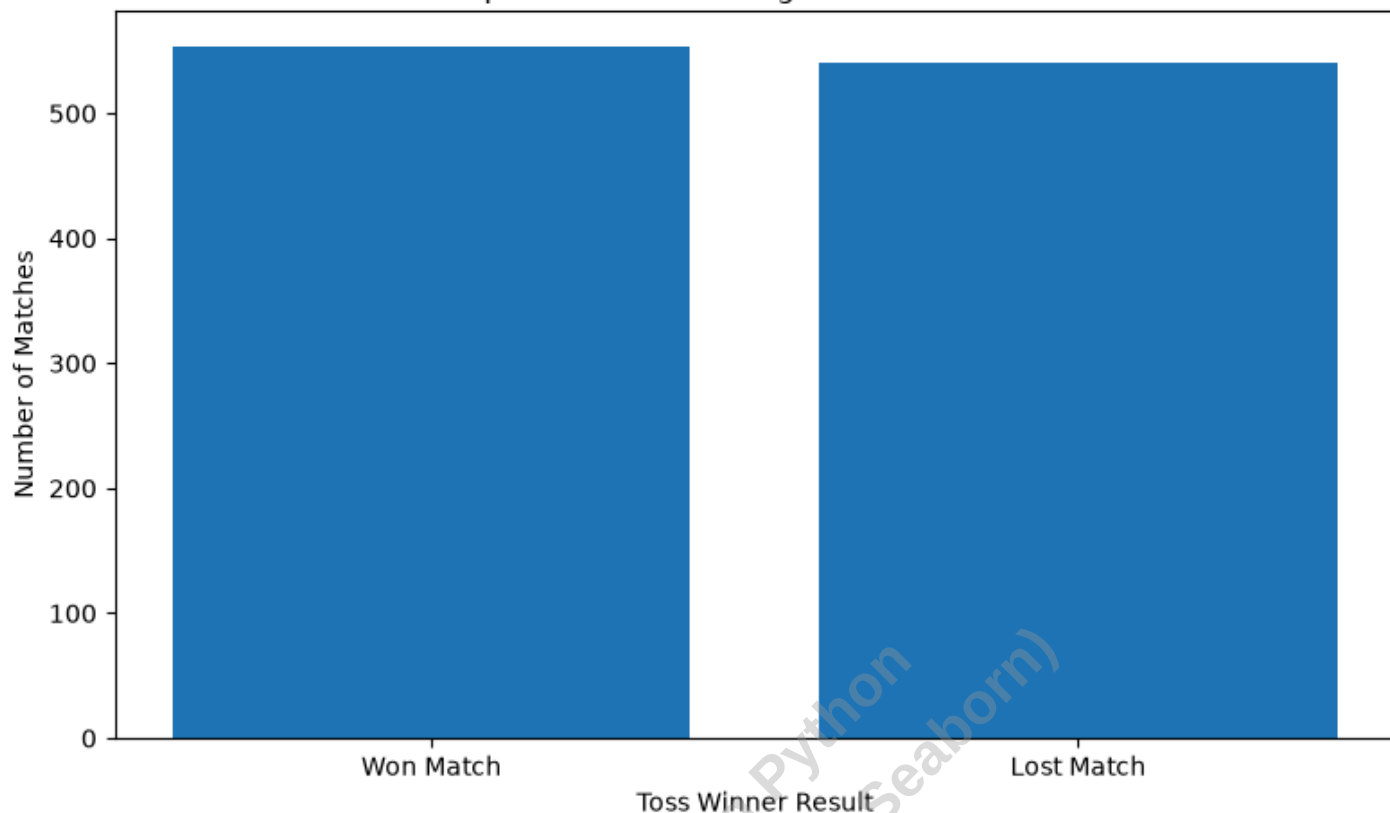
Out[47]:

	Result	Matches
0	Won Match	554
1	Lost Match	541

In [48]: `import matplotlib.pyplot as plt`

```
plt.figure(figsize=(8, 5))  
  
plt.bar(  
    toss_impact.index,  
    toss_impact.values  
)  
  
plt.title('Impact of Toss Winning on Match Results')  
plt.xlabel('Toss Winner Result')  
plt.ylabel('Number of Matches')  
  
plt.tight_layout()  
plt.show()
```

Impact of Toss Winning on Match Results



```
In [49]: toss_percentages = (
    df['toss_match_result']
    .value_counts(normalize=True)
    * 100
)

print(toss_percentages.round(2))
```

```
toss_match_result
Won Match      50.59
Lost Match     49.41
Name: proportion, dtype: float64
```

Insight

The Toss Impact Analysis compares the match outcomes of teams that won the toss.

The analysis shows the percentage of matches in which the toss-winning team also won the match. This helps determine whether winning the toss provides a meaningful advantage in IPL matches.

The results suggest that toss outcomes should be evaluated alongside other factors such as team performance, venue, batting first, and bowling conditions.

```
In [50]: # Step 30: Venue Analysis
```

```
venue_counts = (
    df['venue']
    .value_counts()
    .head(10)
)

venue_counts
```

```
Out[50]: venue
Eden Gardens                77
Wankhede Stadium            73
M Chinnaswamy Stadium       65
Feroz Shah Kotla            60
Rajiv Gandhi International Stadium, Uppal  49
MA Chidambaram Stadium, Chepauk  48
Sawai Mansingh Stadium      47
Dubai International Cricket Stadium  46
Wankhede Stadium, Mumbai    45
Punjab Cricket Association Stadium, Mohali  35
Name: count, dtype: int64
```

```
In [51]: venue_df = venue_counts.reset_index()

venue_df.columns = ['Venue', 'Matches']

venue_df
```

```
Out[51]:
```

	Venue	Matches
0	Eden Gardens	77
1	Wankhede Stadium	73
2	M Chinnaswamy Stadium	65
3	Feroz Shah Kotla	60
4	Rajiv Gandhi International Stadium, Uppal	49
5	MA Chidambaram Stadium, Chepauk	48
6	Sawai Mansingh Stadium	47
7	Dubai International Cricket Stadium	46
8	Wankhede Stadium, Mumbai	45
9	Punjab Cricket Association Stadium, Mohali	35

```
In [52]: import matplotlib.pyplot as plt

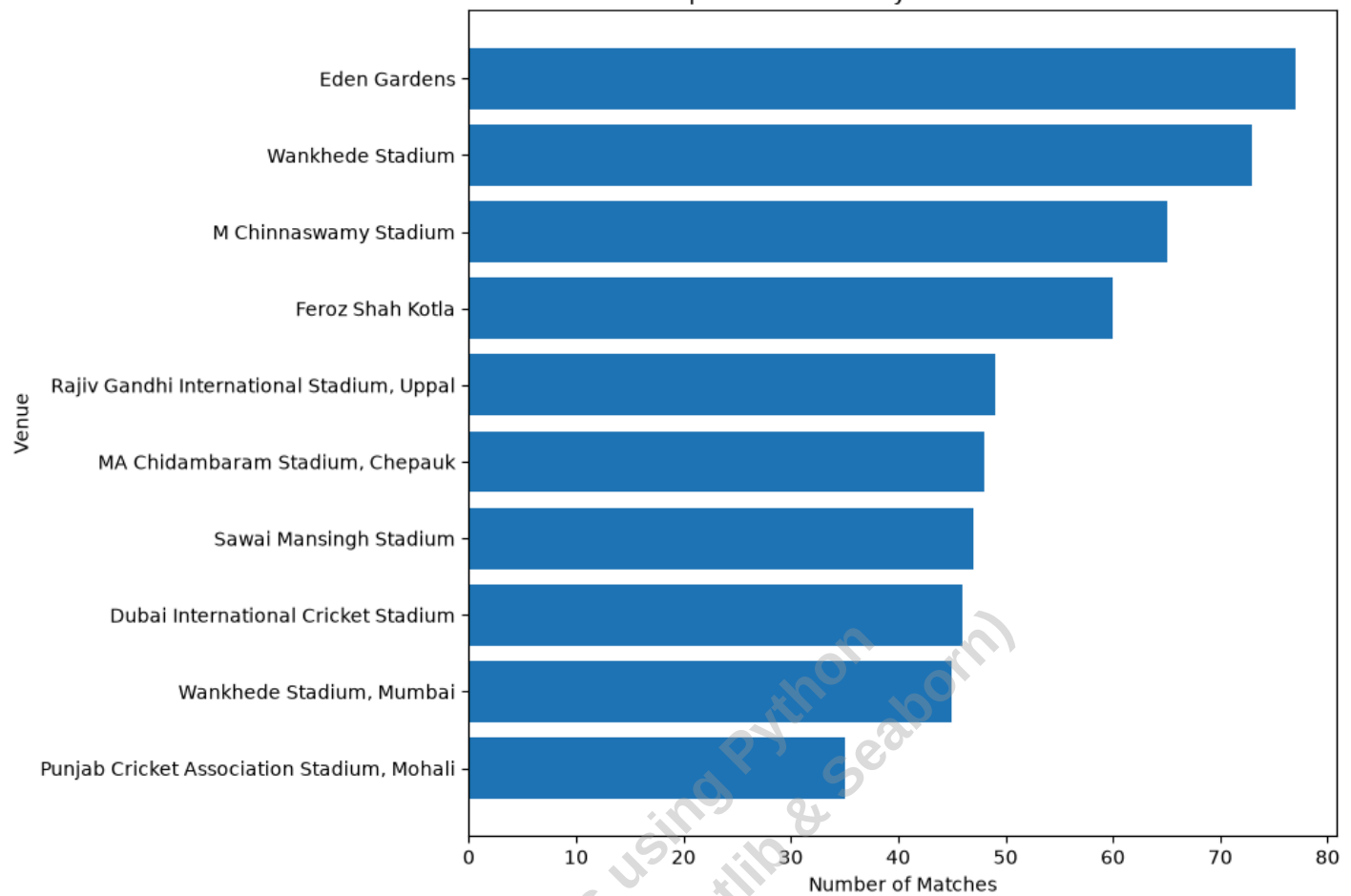
plt.figure(figsize=(10, 7))

plt.barh(
    venue_counts.index[::-1],
    venue_counts.values[::-1]
)

plt.title('Top 10 IPL Venues by Number of Matches')
plt.xlabel('Number of Matches')
plt.ylabel('Venue')

plt.tight_layout()
plt.show()
```

Top 10 IPL Venues by Number of Matches



```
In [53]: top_venue = venue_counts.idxmax()
top_venue_matches = venue_counts.max()

print(f"Venue with the most IPL matches: {top_venue}")
print(f"Number of matches: {top_venue_matches}")
```

Venue with the most IPL matches: Eden Gardens
Number of matches: 77

```
In [54]: venue_percentage = (
    venue_counts / df['venue'].value_counts().sum()
) * 100

venue_percentage
```

```
Out[54]: venue
Eden Gardens                7.031963
Wankhede Stadium            6.666667
M Chinnaswamy Stadium       5.936073
Feroz Shah Kotla            5.479452
Rajiv Gandhi International Stadium, Uppal  4.474886
MA Chidambaram Stadium, Chepauk  4.383562
Sawai Mansingh Stadium      4.292237
Dubai International Cricket Stadium  4.200913
Wankhede Stadium, Mumbai    4.109589
Punjab Cricket Association Stadium, Mohali  3.196347
Name: count, dtype: float64
```

Insight

The Venue Analysis identifies the stadiums that have hosted the highest number of IPL matches.

The most frequently used venue represents an important location in IPL history and reflects the concentration of matches across major cricket venues.

Venue-level analysis can also help identify patterns related to home advantage, match frequency, and stadium utilization.

In [55]: *# Step 31: Season Analysis*

```
season_counts = (  
    df['season']  
    .value_counts()  
    .sort_index()  
)  
  
season_counts
```

Out[55]:

```
season  
2007/08    58  
2009       57  
2009/10    60  
2011       73  
2012       74  
2013       76  
2014       60  
2015       59  
2016       60  
2017       59  
2018       60  
2019       60  
2020/21    60  
2021       60  
2022       74  
2023       74  
2024       71  
Name: count, dtype: int64
```

In [56]:

```
season_df = season_counts.reset_index()  
  
season_df.columns = ['Season', 'Matches']  
  
season_df
```

Out[56]:

	Season	Matches
0	2007/08	58
1	2009	57
2	2009/10	60
3	2011	73
4	2012	74
5	2013	76
6	2014	60
7	2015	59
8	2016	60
9	2017	59
10	2018	60
11	2019	60
12	2020/21	60
13	2021	60
14	2022	74
15	2023	74
16	2024	71

```
In [57]: top_season = season_counts.idxmax()
top_season_matches = season_counts.max()

print(f"Season with the most matches: {top_season}")
print(f"Number of matches: {top_season_matches}")
```

Season with the most matches: 2013
Number of matches: 76

```
In [58]: import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

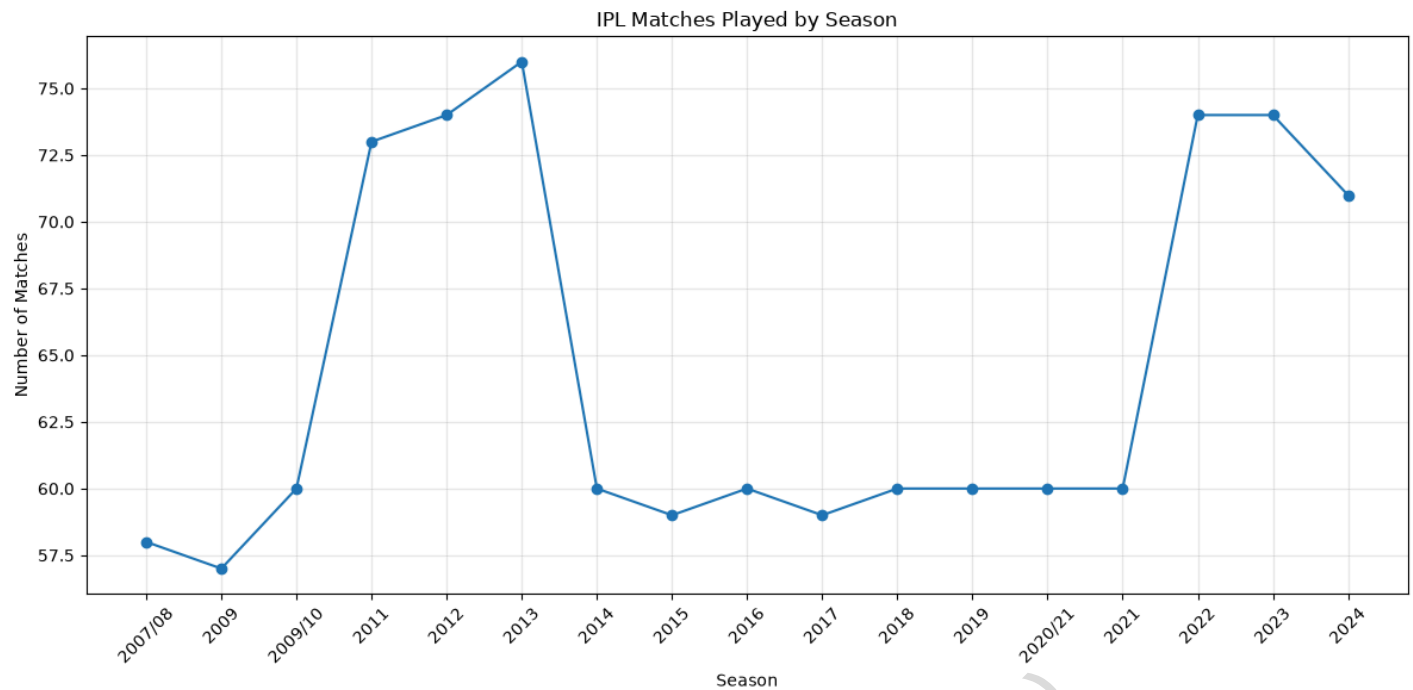
plt.plot(
    season_counts.index,
    season_counts.values,
    marker='o'
)

plt.title('IPL Matches Played by Season')
plt.xlabel('Season')
plt.ylabel('Number of Matches')

plt.xticks(rotation=45)

plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
In [59]: import pandas as pd
```

```
In [60]: import os
```

```
for root, dirs, files in os.walk(".."):
    for file in files:
        if file.lower().endswith(".csv"):
            print(os.path.join(root, file))
```

```
..\data\deliveries.csv
..\data\matches.csv
..\venv\Lib\site-packages\matplotlib\mpl-data\sample_data\data_x_x2_x3.csv
..\venv\Lib\site-packages\matplotlib\mpl-data\sample_data\msft.csv
..\venv\Lib\site-packages\matplotlib\mpl-data\sample_data\Stocks.csv
..\venv\Lib\site-packages\numpy\random\tests\data\mt19937-testset-1.csv
..\venv\Lib\site-packages\numpy\random\tests\data\mt19937-testset-2.csv
..\venv\Lib\site-packages\numpy\random\tests\data\pcg64-testset-1.csv
..\venv\Lib\site-packages\numpy\random\tests\data\pcg64-testset-2.csv
..\venv\Lib\site-packages\numpy\random\tests\data\pcg64xsm-testset-1.csv
..\venv\Lib\site-packages\numpy\random\tests\data\pcg64xsm-testset-2.csv
..\venv\Lib\site-packages\numpy\random\tests\data\philox-testset-1.csv
..\venv\Lib\site-packages\numpy\random\tests\data\philox-testset-2.csv
..\venv\Lib\site-packages\numpy\random\tests\data\sfc64-testset-1.csv
..\venv\Lib\site-packages\numpy\random\tests\data\sfc64-testset-2.csv
```

```
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-arccos.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-arccosh.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-arcsin.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-arcsinh.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-arctan.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-arctanh.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-cbrt.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-cos.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-cosh.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-exp.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-exp2.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-expm1.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-log.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-log10.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-log1p.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-log2.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-sin.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-sinh.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-tan.csv
..\venv\Lib\site-packages\numpy\_core\tests\data\umath-validation-set-tanh.csv
..\venv\Lib\site-packages\tornado\test\csv_translations\fr_FR.csv
```

In [61]: `import pandas as pd`

```
df = pd.read_csv("../data/matches.csv")

print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Dataset loaded successfully!

Rows: 1095

Columns: 20

In [62]: `print(df.head())`

	id	season	city	date	match_type	player_of_match	\
0	335982	2007/08	Bangalore	2008-04-18	League	BB McCullum	
1	335983	2007/08	Chandigarh	2008-04-19	League	MEK Hussey	
2	335984	2007/08	Delhi	2008-04-19	League	MF Maharooof	
3	335985	2007/08	Mumbai	2008-04-20	League	MV Boucher	
4	335986	2007/08	Kolkata	2008-04-20	League	DJ Hussey	

		venue	team1	\
0		M Chinnaswamy Stadium	Royal Challengers Bangalore	
1	Punjab Cricket Association Stadium, Mohali		Kings XI Punjab	
2		Feroz Shah Kotla	Delhi Daredevils	
3		Wankhede Stadium	Mumbai Indians	
4		Eden Gardens	Kolkata Knight Riders	

	team2	toss_winner	toss_decision	\
0	Kolkata Knight Riders	Royal Challengers Bangalore	field	
1	Chennai Super Kings	Chennai Super Kings	bat	
2	Rajasthan Royals	Rajasthan Royals	bat	
3	Royal Challengers Bangalore	Mumbai Indians	bat	
4	Deccan Chargers	Deccan Chargers	bat	

	winner	result	result_margin	target_runs	\
0	Kolkata Knight Riders	runs	140.0	223.0	
1	Chennai Super Kings	runs	33.0	241.0	
2	Delhi Daredevils	wickets	9.0	130.0	
3	Royal Challengers Bangalore	wickets	5.0	166.0	
4	Kolkata Knight Riders	wickets	5.0	111.0	

	target_overs	super_over	method	umpire1	umpire2
0	20.0	N	NaN	Asad Rauf	RE Koertzen
1	20.0	N	NaN	MR Benson	SL Shastri
2	20.0	N	NaN	Aleem Dar	GA Pratapkumar
3	20.0	N	NaN	SJ Davis	DJ Harper
4	20.0	N	NaN	BF Bowden	K Hariharan

```
In [63]: print(df.columns.tolist())
```

```
['id', 'season', 'city', 'date', 'match_type', 'player_of_match', 'venue', 'team1', 'team2', 'toss_winner', 'toss_decision', 'winner', 'result', 'result_margin', 'target_runs', 'target_overs', 'super_over', 'method', 'umpire1', 'umpire2']
```

```
In [64]: # Step 31 - Season Analysis
```

```
season_df = (
    df.groupby('season')
      .size()
      .reset_index(name='Matches')
)

season_df['Change from Previous Season'] = (
    season_df['Matches'].diff()
)

season_df['Growth %'] = (
    season_df['Matches'].pct_change() * 100
)

season_df
```

Out[64]:

	season	Matches	Change from Previous Season	Growth %
0	2007/08	58	NaN	NaN
1	2009	57	-1.0	-1.724138
2	2009/10	60	3.0	5.263158
3	2011	73	13.0	21.666667
4	2012	74	1.0	1.369863
5	2013	76	2.0	2.702703
6	2014	60	-16.0	-21.052632
7	2015	59	-1.0	-1.666667
8	2016	60	1.0	1.694915
9	2017	59	-1.0	-1.666667
10	2018	60	1.0	1.694915
11	2019	60	0.0	0.000000
12	2020/21	60	0.0	0.000000
13	2021	60	0.0	0.000000
14	2022	74	14.0	23.333333
15	2023	74	0.0	0.000000
16	2024	71	-3.0	-4.054054

In [65]:

```
import matplotlib.pyplot as plt

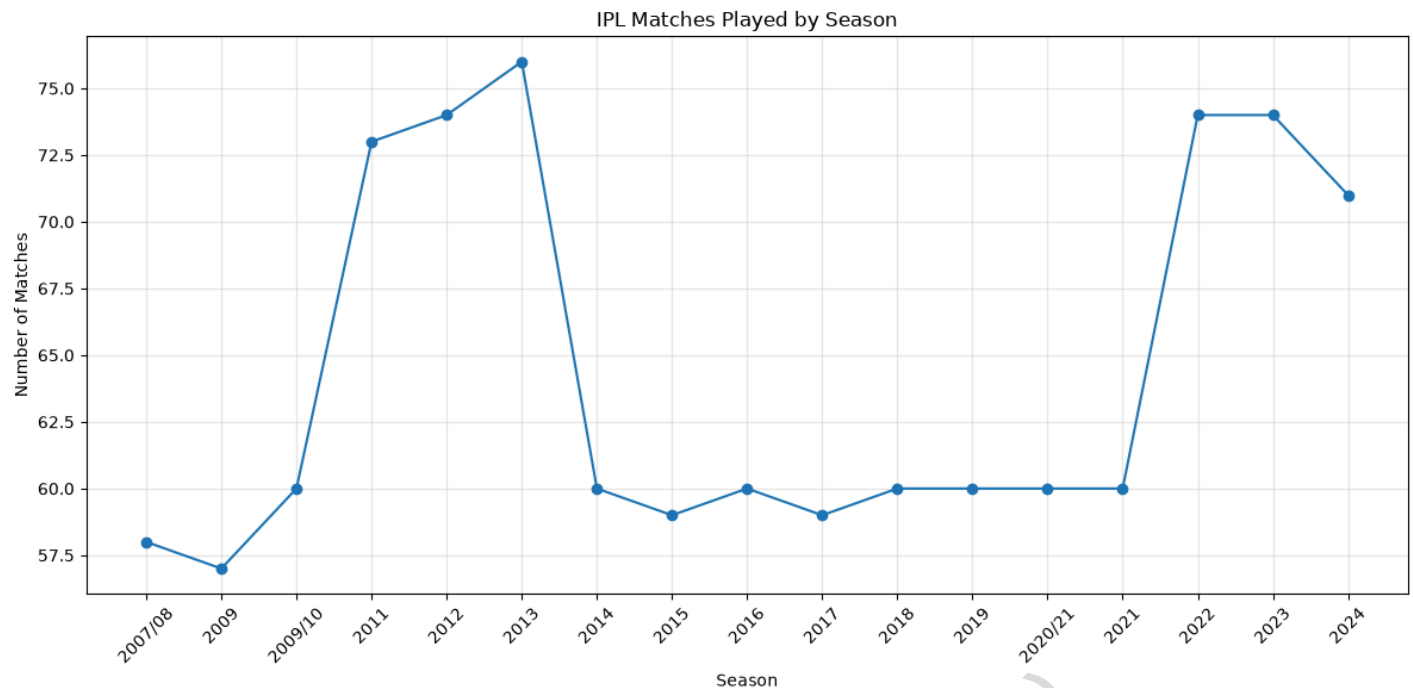
plt.figure(figsize=(12, 6))

plt.plot(
    season_df['season'],
    season_df['Matches'],
    marker='o'
)

plt.title('IPL Matches Played by Season')
plt.xlabel('Season')
plt.ylabel('Number of Matches')

plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In []:

Insight

The Season Analysis shows how the IPL tournament has evolved over time based on the number of matches played in each season.

The season-wise trend highlights changes in tournament size and structure across different years. Changes in the number of matches can reflect differences in the number of participating teams, tournament format, and scheduling.

The analysis demonstrates the growth and evolution of the IPL as a major cricket tournament.

```
In [66]: # =====
# STEP 32 - FINAL DATA INSIGHTS
# =====

import pandas as pd

# Load datasets
df = pd.read_csv("../data/matches.csv")
deliveries = pd.read_csv("../data/deliveries.csv")

# -----
# 1. Team Performance
# -----

team_wins = df["winner"].dropna().value_counts()

top_team = team_wins.index[0]
top_team_wins = team_wins.iloc[0]

# -----
# 2. Top Batsman
# -----

top_batsmen = (
```

```

        deliveries.groupby("batter")["batsman_runs"]
        .sum()
        .sort_values(ascending=False)
    )

top_batsman = top_batsmen.index[0]
top_batsman_runs = top_batsmen.iloc[0]

# -----
# 3. Top Bowler
# -----

valid_dismissals = [
    "run out",
    "retired hurt",
    "obstructing the field"
]

bowler_wickets = (
    deliveries[
        deliveries["dismissal_kind"].notna()
        & ~deliveries["dismissal_kind"].isin(valid_dismissals)
    ]
    .groupby("bowler")
    .size()
    .sort_values(ascending=False)
)

top_bowler = bowler_wickets.index[0]
top_bowler_wickets = bowler_wickets.iloc[0]

# -----
# 4. Player of the Match
# -----

pom_counts = df["player_of_match"].dropna().value_counts()

top_pom_player = pom_counts.index[0]
top_pom_awards = pom_counts.iloc[0]

# -----
# 5. Toss Impact
# -----

toss_matches = df[
    df["toss_winner"].notna()
    & df["winner"].notna()
]

toss_winner_match_wins = (
    toss_matches["toss_winner"] == toss_matches["winner"]
).sum()

toss_win_percentage = (
    toss_winner_match_wins / len(toss_matches)
) * 100

# -----
# 6. Top Venue

```

```

# -----

venue_counts = df["venue"].dropna().value_counts()

top_venue = venue_counts.index[0]
top_venue_matches = venue_counts.iloc[0]

# -----
# 7. Top Season
# -----

season_counts = df["season"].value_counts()

top_season = season_counts.index[0]
top_season_matches = season_counts.iloc[0]

# =====
# FINAL INSIGHTS TABLE
# =====

final_insights = pd.DataFrame({
    "Analysis": [
        "Team Performance",
        "Top Batsman",
        "Top Bowler",
        "Player of the Match",
        "Toss Impact",
        "Top Venue",
        "Top Season"
    ],
    "Key Finding": [
        f"{top_team} ({top_team_wins} wins)",
        f"{top_batsman} ({top_batsman_runs} runs)",
        f"{top_bowler} ({top_bowler_wickets} wickets)",
        f"{top_pom_player} ({top_pom_awards} awards)",
        f"{toss_win_percentage:.2f}% toss winners also won",
        f"{top_venue} ({top_venue_matches} matches)",
        f"{top_season} ({top_season_matches} matches)"
    ]
})

final_insights

```

Out[66]:

	Analysis	Key Finding
0	Team Performance	Mumbai Indians (144 wins)
1	Top Batsman	V Kohli (8014 runs)
2	Top Bowler	YS Chahal (205 wickets)
3	Player of the Match	AB de Villiers (25 awards)
4	Toss Impact	50.83% toss winners also won
5	Top Venue	Eden Gardens (77 matches)
6	Top Season	2013 (76 matches)

In [67]: # Step 32: Final Data Insights

```
print("==== IPL DATA ANALYSIS – FINAL INSIGHTS =====\n")

print("1. Team Performance")
print(f"Most successful team by wins: {team_wins.index[0]}")
print(f"Total wins: {team_wins.iloc[0]}\n")

print("2. Top Batsman")
print(f"Top batsman: {top_batsmen.index[0]}")
print(f"Runs: {top_batsmen.iloc[0]}\n")

print("3. Top Bowler")
print(f"Top bowler: {top_bowlers.index[0]}")
print(f"Wickets: {top_bowlers.iloc[0]}\n")

print("4. Player of the Match")
print(f"Leading player: {top_pom_player}")
print(f"Awards: {top_pom_awards}\n")

print("5. Toss Impact")
print(f"Toss winner also won the match: {toss_win_percentage:.2f}%\n")

print("6. Venue Analysis")
print(f"Most-used venue: {top_venue}")
print(f"Matches hosted: {top_venue_matches}\n")

print("7. Season Analysis")
print(f"Season with most matches: {top_season}")
print(f"Matches: {top_season_matches}")
```

==== IPL DATA ANALYSIS – FINAL INSIGHTS =====

1. Team Performance
Most successful team by wins: Mumbai Indians
Total wins: 144

2. Top Batsman
Top batsman: V Kohli
Runs: 8014

3. Top Bowler
Top bowler: R Ashwin
Wickets: 4661

4. Player of the Match
Leading player: AB de Villiers
Awards: 25

5. Toss Impact
Toss winner also won the match: 50.83%

6. Venue Analysis
Most-used venue: Eden Gardens
Matches hosted: 77

7. Season Analysis
Season with most matches: 2013
Matches: 76

In [68]: # =====
STEP 32 - FINAL DATA INSIGHTS
=====

```

import pandas as pd

# Load IPL datasets
df = pd.read_csv("../data/matches.csv")
deliveries = pd.read_csv("../data/deliveries.csv")

# =====
# 1. TEAM PERFORMANCE
# =====

team_wins = df["winner"].dropna().value_counts()

top_team = team_wins.index[0]
top_team_wins = team_wins.iloc[0]

# =====
# 2. TOP BATSMAN
# =====

top_batsmen = (
    deliveries
    .groupby("batter")["batsman_runs"]
    .sum()
    .sort_values(ascending=False)
)

top_batsman = top_batsmen.index[0]
top_batsman_runs = top_batsmen.iloc[0]

# =====
# 3. TOP BOWLER
# =====

excluded_dismissals = [
    "run out",
    "retired hurt",
    "obstructing the field"
]

bowler_wickets = (
    deliveries[
        deliveries["dismissal_kind"].notna()
        & ~deliveries["dismissal_kind"].isin(excluded_dismissals)
    ]
    .groupby("bowler")
    .size()
    .sort_values(ascending=False)
)

top_bowler = bowler_wickets.index[0]
top_bowler_wickets = bowler_wickets.iloc[0]

# =====
# 4. PLAYER OF THE MATCH
# =====

pom_counts = (
    df["player_of_match"]
    .dropna()

```

```

        .value_counts()
    )

    top_pom_player = pom_counts.index[0]
    top_pom_awards = pom_counts.iloc[0]

# =====
# 5. TOSS IMPACT
# =====

toss_data = df[
    df["toss_winner"].notna()
    & df["winner"].notna()
]

toss_wins_match = (
    toss_data["toss_winner"] == toss_data["winner"]
).sum()

toss_win_percentage = (
    toss_wins_match / len(toss_data)
) * 100

# =====
# 6. TOP VENUE
# =====

venue_counts = df["venue"].dropna().value_counts()

top_venue = venue_counts.index[0]
top_venue_matches = venue_counts.iloc[0]

# =====
# 7. TOP SEASON
# =====

season_counts = df["season"].value_counts()

top_season = season_counts.index[0]
top_season_matches = season_counts.iloc[0]

# =====
# 8. FINAL INSIGHTS TABLE
# =====

final_insights = pd.DataFrame({
    "Analysis": [
        "Team Performance",
        "Top Batsman",
        "Top Bowler",
        "Player of the Match",
        "Toss Impact",
        "Top Venue",
        "Top Season"
    ],
    "Key Finding": [
        f"{top_team} - {top_team_wins} wins",
        f"{top_batsman} - {top_batsman_runs} runs",

```

```

f"{top_bowler} - {top_bowler_wickets} wickets",
f"{top_pom_player} - {top_pom_awards} awards",
f"{toss_win_percentage:.2f}% toss winners also won",
f"{top_venue} - {top_venue_matches} matches",
f"{top_season} - {top_season_matches} matches"
]
})

final_insights

```

Out[68]:

	Analysis	Key Finding
0	Team Performance	Mumbai Indians - 144 wins
1	Top Batsman	V Kohli - 8014 runs
2	Top Bowler	YS Chahal - 205 wickets
3	Player of the Match	AB de Villiers - 25 awards
4	Toss Impact	50.83% toss winners also won
5	Top Venue	Eden Gardens - 77 matches
6	Top Season	2013 - 76 matches



Final IPL Data Insights

Key Findings



Team Performance

The team-performance analysis identifies the teams that have achieved the highest number of wins across the dataset.



Individual Performance

The batsmen and bowlers analysis highlights the leading individual performers based on runs and wickets.



Player of the Match

The Player of the Match analysis identifies the players who have received the greatest number of individual match-performance awards.

Toss Impact

The toss analysis measures how frequently the team winning the toss also wins the match. This provides an indication of whether the toss is associated with match success.



Venue Analysis

The venue analysis identifies the stadiums that have hosted the largest number of IPL matches.



Season Analysis

The season analysis demonstrates how the number of IPL matches has changed across seasons and provides an overview of the tournament's evolution.

Overall Conclusion

The IPL dataset provides valuable insights into team performance, individual player performance, toss outcomes, venue distribution, and season-wise tournament trends.

Combining these analyses demonstrates how Python and data visualization can be used to transform historical cricket data into meaningful, measurable insights.

IPL Data Analysis using Python
(NumPy, Pandas, Matplotlib & Seaborn)